

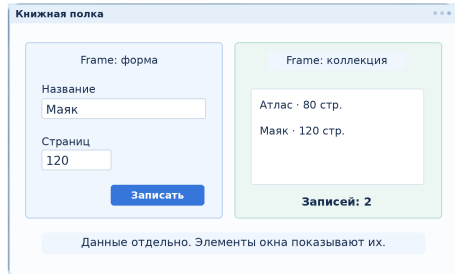
ОСНОВАНИЕ · PYTHON

Tkinter с нуля

От первого окна
к небольшому приложению

Теория и примеры к 20 задачам

Дмитрий Нефедов



От идеи к окну

Не заучиваем интерфейсы. Учимся
связывать данные, действия и ре-
зультат.

Маршрут: от окна к данным

01–04 · Окно отвечает

Окно, надпись, кнопка, состояние, первое поле.

05–10 · Приложение считает

Числа, несколько действий, ошибки ввода и память.

11–13 · Пользователь выбирает

Флажки, радиокнопки, ползунок.

14–16 · Данные становятся длиннее

Список, выделение, удаление, многострочный текст.

17–20 · Собираем приложение

Комбинация элементов, группы, порядок записей и сводка.

Все номера здесь — **внутри серии Tkinter**.
Восемь задач-разминок на функции не входят в эту нумерацию.

В окне порядок действий задаёт пользователь

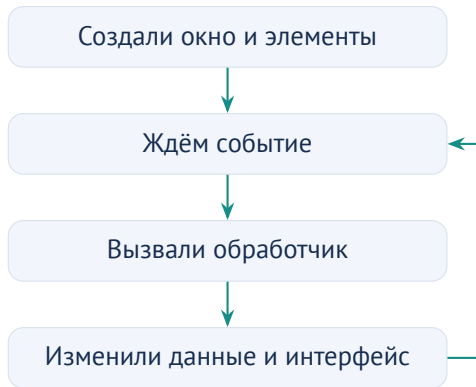
Консольная программа

Получить ввод → вычислить → напечатать → завершиться.

Оконная программа

Создать интерфейс → ждать действий → отвечать на каждое действие.

`mainloop()` поддерживает обработку событий и обновление окна.



Не ставьте длительный цикл или `sleep()` в обработчик: окно перестанет вовремя отвечать.

Первое окно: создать, настроить, запустить

Полная программа · examples/01_window.py

```
import tkinter as tk

root = tk.Tk()
root.title("Лаборатория")
root.geometry("480x300")
out = tk.Label(root,
               text="Можно начинать")
out.pack(pady=20)

root.mainloop()
```



Tk — главное окно.
title() — его заголовок.
geometry("480x300") — ширина и высота рабочей области.

Проверка установки: `python -m tkinter`; на Windows также `py -m tkinter`.

Виджет — отдельный элемент интерфейса

У каждого виджета есть

Тип: что он умеет.

Родитель: где он находится.

Параметры: как выглядит и работает.

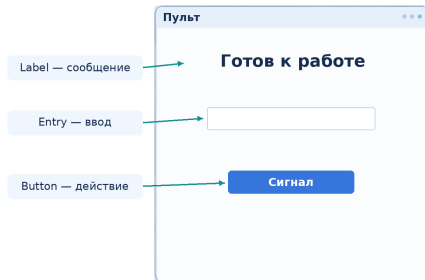
Три первых элемента

Label показывает сообщение.

Entry принимает одну строку.

Button запускает действие.

Создать виджет и разместить его — **два разных действия**.



Label: показать и изменить текст

Фрагмент · root уже создан

```
out = tk.Label(root, text="Статус", font=("Arial", 16))
out.pack(pady=12)

out.config(text="Сигнал принят")
out["text"] = "Пауза"
s = out.cget("text")
```

Запись

`config()` и `configure()` – настройка параметров. Запись `out["text"] = ...` тоже меняет текст.

Чтение

`cget("text")` возвращает текущее значение параметра. В этом примере в `s` будет строка «Пауза».

Если надпись уже есть, изменяйте её. Новая Label при каждом клике создаёт лишние элементы.

pack: простая последовательность элементов

Фрагмент · root уже создан

```
name = tk.Label(root, text="Название")
entry = tk.Entry(root)
name.pack(pady=8)
entry.pack(fill="x", padx=20, pady=8)
```

Положение

По умолчанию элементы идут сверху вниз. `side="left"` размещает вдоль левого края.

Воздух и размер

`padx, pady` – внешние отступы.
`fill="x"` растягивает элемент по ширине доступной области.

Не пишите `entry = tk.Entry(root).pack(): pack()` возвращает `None`, а не созданное поле.

grid: форма как таблица

Фрагмент · root уже создан; в этом контейнере используется только grid

```
tk.Label(root, text="Книга").grid(row=0, column=0, sticky="w")
entry = tk.Entry(root)
entry.grid(row=0, column=1, sticky="ew")

tk.Label(root, text="Автор").grid(row=1, column=0, sticky="w")
author = tk.Entry(root)
author.grid(row=1, column=1, sticky="ew")
root.columnconfigure(1, weight=1)
```

Координаты

row — строка, column — столбец. Нумерация начинается с нуля.

Выравнивание

sticky="w" — влево; "ew" — растянуть по ширине ячейки. weight=1 разрешает столбцу получать свободное место.

Frame: группа внутри окна

Фрагмент · root уже создан

```
form = tk.Frame(root)
form.pack(padx=16, pady=16)

label = tk.Label(form, text="Книга")
label.grid(row=0, column=0)
entry = tk.Entry(form)
entry.grid(row=0, column=1)
```

Frame не открывает новое окно. Это контейнер для других виджетов.

Дерево родителей

root

↪ form

↪ Label

↪ Entry

Почему это работает

Frame размещён через pack. Его дети размещены через grid. Родители различаются.

Не смешивайте pack и grid **у детей одного контейнера**. Между разными Frame способы могут отличаться.

Button: действие по нажатию

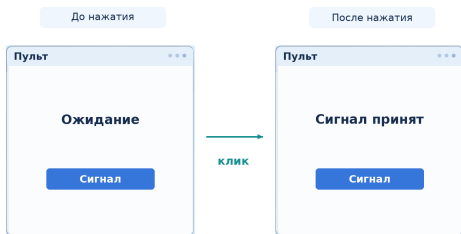
Фрагмент · root и надпись out уже созданы

```
def signal():  
    out.config(text="Сигнал принят")  
  
btn = tk.Button(root, text="Сигнал",  
                command=signal)  
btn.pack()
```

Обработчик, или callback

Обычная функция. Tkinter вызовет её, когда пользователь нажмёт кнопку.

Кнопка получает **функцию**, а не результат её выполнения.



command: функция сейчас, вызов потом

Три записи — три разных смысла

Фрагменты · signal, root и out существуют

```
tk.Button(root, text="Сигнал", command=signal)
```

Передали функцию. Она будет вызвана **при нажатии**.

```
tk.Button(root, text="Сигнал", command=signal())
```

Вызвали функцию **при создании кнопки**. В command попал её результат.

```
def choose(lang):  
    out.config(text=lang)
```

```
btn = tk.Button(root, text="RU", command=lambda: choose("ru"))
```

Подумайте. Почему после lambda: вызов со скобками уже допустим?

Мини-пример: меняем существующую надпись

Полная программа · [examples/02_signal.py](#)

```
import tkinter as tk

root = tk.Tk()
root.title("Пульт")
root.geometry("480x300")
out = tk.Label(root, text="Ожидание")
out.pack(pady=20)

def signal():
    out.config(text="Сигнал принят")

tk.Button(root, text="Сигнал", command=signal).pack()
root.mainloop()
```

До `mainloop()` мы собираем интерфейс. В `signal()` описываем ответ на будущий клик.

Состояние должно переживать отдельный клик

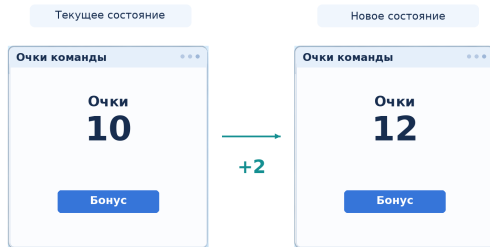
Фрагмент · out уже создан; начальная надпись показывает

10

```
n = 10

def bonus():
    global n
    n += 2
    out.config(text=f"Очки: {n}")
```

n создан один раз, вне обработчика.
global n разрешает изменить переменную модуля внутри функции.



Если присваивать `n = 10` внутри `bonus()`, каждый клик начнёт вычисление заново.

Переменные Tkinter связывают данные и виджеты

Фрагмент · root уже создан

```
status = tk.StringVar(root, value="В пути")
out = tk.Label(root, textvariable=status)
out.pack()
```

```
status.set("На месте")
s = status.get()
```

`textvariable=status` связывает надпись с объектом. `status.set(...)` обновляет связанный текст.

StringVar

Строка: подпись, поле, выбранный режим.

IntVar

Целое число: счётчик или числовой выбор.

BooleanVar

Логическое состояние: включено или выключено.

Читать значение — `get()`. Менять значение — `set(...)`. Обычная переменная Python сама по себе с виджетом не связана.

Entry: на экране цифры, внутри – строка

Фрагмент · root и out уже созданы

```
entry = tk.Entry(root)
entry.pack()
```

```
def read():
    raw = entry.get()
    n = len(raw)
    out.config(text=f"Длина: {n}")
```

Что вернёт get()

Всегда строку: "42", " Марс " или пустую "".

Строка из поля

« Марс »

strip()

«Марс»

Обработка строки не изменяет само поле

`raw.strip()` создаёт строку без пробелов по краям. Сам Entry при этом не изменяется.

Считывайте поле в момент действия

Фрагмент · root, entry и out созданы

```
def read():  
    raw = entry.get()  
    clean = raw.strip()  
    out.config(text=clean)
```

Новое чтение при каждом клике

Пользователь изменил поле — следующий клик увидит новое значение.

Сравните с этим вариантом

```
raw = entry.get()  
  
def read():  
    out.config(text=raw)
```

Сохранили старый снимок

Здесь raw получен один раз. Сам по себе он больше не обновляется.

Подумайте. В поле было «Лес», потом ввели «Море». Что покажет каждый вариант после клика?

Изменить поле: удалить, затем вставить

Фрагмент · entry — уже созданное поле

```
old = entry.get()
entry.delete(0, tk.END)
entry.insert(0, "012")
```

Индексы Entry

Первый символ имеет индекс 0.
`delete(0, tk.END)` очищает поле.
`insert(0, text)` вставляет с начала.

Два поля: сначала снимок

Сначала прочитайте **оба** исходных текста. Только потом меняйте поля. Иначе одно значение можно потерять.

Если нужно сохранить запись «012», не превращайте её в число. `int("012")` даёт число 12.

Числа и ошибки: не закрываем окно

Фрагмент · entry и out созданы; функция назначается кнопке

```
def check():  
    try:  
        n = int(entry.get())  
    except ValueError:  
        out.config(text="Нужно целое число")  
        return  
    out.config(text=f"Квадрат: {n * n}")
```

Преобразование

`int(" -7 ")` допустимо. `int("")` и `int("2.5")` вызывают `ValueError`.

Восстановление

Показали ошибку → вышли из обработчика. Пользователь исправит ввод и нажмёт снова.

Несколько полей — один последовательный расчёт

Фрагмент · два Entry width/height и out созданы; ввод здесь корректный

```
def area():  
    a = int(width.get())  
    b = int(height.get())  
    out["text"] = f"Площадь: {a * b}"
```

Четыре шага

1. Считать оба поля.
2. Преобразовать значения.
3. Вычислить результат.
4. Обновить надпись.

Другая кнопка может использовать те же поля, но выполнять другую операцию. Поля сами от этого не меняются.

Площадь открытки

Ширина, см

Высота, см

Площадь

28 см²

Поле, результат и память — разные данные

Ввод

Текущий текст пользователя. Может ещё не быть числом.

Результат

Число последнего успешного действия или состояние «нет результата».

Снимок

Сохранённое значение. Оно не меняется само при редактировании полей.

```
saved = None
```

```
saved = 0
```

```
available = saved is not None
```

Ноль — значение, None — отсутствие

Проверка `if saved:` не различает нулевой результат и отсутствие данных. Проверка `saved is not None` различает.

Для каждого действия заранее решите: что оно изменяет и что обязано сохранить. Не храните все состояния в одной строке Label.

Пауза: найдите три причины странного поведения

Мини-разбор · не исправляйте сразу: сначала объясните причину

```
btn = tk.Button(root, text="Тест", command=check())
```

А. Почему действие происходит до первого нажатия?

```
entry = tk.Entry(root).pack()
```

В. Почему после этого у entry нет метода get?

```
n = 0

def click():
    n = 0
    n += 1
```

С. Почему значение не накапливается?

Подумайте. Назовите одну минимальную правку для каждого случая.

Checkbox: несколько независимых опций

Фрагмент · root уже создан

```
pages = tk.BooleanVar(root, value=True)
box = tk.Checkbutton(
    root, text="Номера страниц", variable=pages
)
box.pack()

is_on = pages.get()
```

Связь с данными

Флажок меняет связанную переменную. `pages.get()` возвращает `True` или `False`.

Независимость

Разные независимые флажки — разные переменные. Можно включить ни одного, один или несколько.

Нужна кнопка подтверждения? Читайте флажки в её обработчике, а не обновляйте итог при каждом переключении.

Radiobutton: общий выбор, разные значения

Фрагмент · root уже создан

```
mode = tk.StringVar(root, value="light")
tk.Radiobutton(root, text="Белая",
               variable=mode, value="light").pack()
tk.Radiobutton(root, text="Сепия",
               variable=mode, value="sepia").pack()

current = mode.get()
```

Одна группа

Все варианты используют **одну** переменную `mode`. Поэтому выбирается один вариант из группы.

Разные value

Текст «Сепия» видит пользователь. Значение `"sepia"` хранится в программе. Это не обязательно быть одной строкой.

Начальное значение переменной должно совпадать с `value` нужного варианта. Иначе изначально ни один не будет выбран.

Выбрано сейчас $\neq$ применено раньше

Настройки читалки

Несколько независимых

Один из вариантов

☒ Номера страниц

☐ Подсветка

☐ Белая

☒ Сепия

Текущее состояние

Меняется, когда пользователь переключает элементы.

Подтверждённый результат

Меняется только по кнопке, если так сказано в условии.

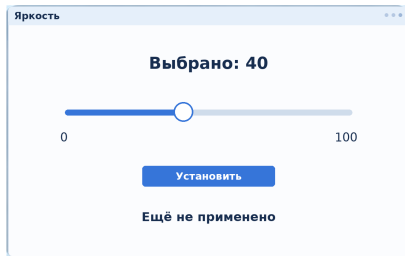
Фрагмент · `mode`, `pages` и `out` уже созданы

```
def apply():  
    current_mode = mode.get()  
    show_pages = pages.get()  
    out.config(text=f"{current_mode} / {show_pages}")
```

Scale: число из диапазона

Фрагмент · root и out созданы

```
level = tk.Scale(
    root, from_=0, to=100,
    resolution=10, orient="horizontal"
)
level.set(40)
level.pack()
def apply():
    n = level.get()
    out.config(text=f"Выбрано: {n}")
```



from_, to – границы.
resolution – шаг.
set() – установить.
get() – прочитать.

В контексте: Scale читаем **отдельной кнопкой**, без command у ползунка.

Составная форма: снимите параметры, потом считайте

Фрагмент · поля и переменные формы существуют

```
name = entry.get().strip()  
current = mode.get()  
is_on = pages.get()
```

Снимок формы

Текст, выбранный вариант и флажок считываются в начале одного действия.

Проверка

Нужные поля не пусты? Значения допустимы? При ошибке покажите сообщение и оставьте возможность исправиться.

Расчёт

Вычислите итог заново по текущим параметрам. Например: базовое значение и выбранные дополнения.

Повторный клик с теми же данными должен давать тот же итог, если действие не является накоплением.

Listbox: список отдельных строк

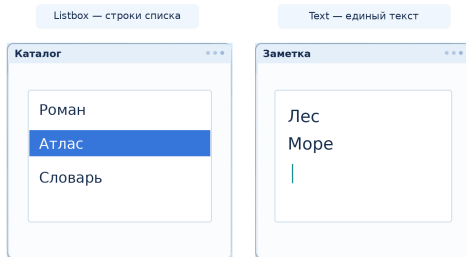
Фрагмент · root уже создан

```
lst = tk.Listbox(root,  
    selectmode="single",  
    exportselection=False)  
lst.pack()  
lst.insert(tk.END, "Роман")  
lst.insert(tk.END, "Атлас")  
count = lst.size()
```

`insert(tk.END, text)` — добавить в конец.
`size()` — число элементов.

Одинаковые строки могут встречаться несколько раз.

Listbox хранит строки по позициям. Text, который разберём дальше, хранит единый редактируемый текст.



Выделение — это кортеж индексов

Фрагмент · lst и out уже созданы

```
def show_selected():  
    sel = lst.curselection()  
    if not sel:  
        return  
    i = sel[0]  
    out.config(text=lst.get(i))
```

Нет выбора

lst.curselection() → ()
Сначала проверяем пустой кортеж.

Выбрана первая строка

Результат — (0,).
Кортеж непустой, а индекс равен нулю.

if i: пропустит первую строку. Проверяйте сам кортеж выбора, а затем используйте sel[0].

lst.get(i) возвращает одну строку; lst.get(0, tk.END) — кортеж всех строк.

Удаление: выбор, действие, новое выделение

Фрагмент · lst уже создан

```
def remove():  
    sel = lst.curselection()  
    if not sel:  
        return  
    lst.delete(sel[0])  
    lst.selection_clear(0, tk.END)
```

Правило границ Listbox

`delete(i)` удаляет один элемент.
`delete(0, tk.END)` удаляет все.
Правая граница диапазона включается.

Проверяйте последствия

Порядок оставшихся строк.
Счётчик элементов.
Состояние выделения.
Поле ввода — только по условию.

Меняем порядок: элемент и его индекс — не одно

До

0 · Роман

1 · **Атлас** ← **выбран**

2 · Словарь

После перемещения

0 · **Атлас** ← **выбран**

1 · Роман

2 · Словарь

Фрагмент · a — Python-список, i — выбранный индекс

```
j = i - 1
if 0 <= j < len(a):
    a[i], a[j] = a[j], a[i]
```

После изменения данных перерисуйте список и восстановите выбор по **новому** индексу. Число элементов не меняется.

Индекс -1 в Python допустим и означает последний элемент. Поэтому границы проверяем **до** обмена.

Text: строки и столбцы вместо одного индекса

Фрагмент · root уже создан

```
editor = tk.Text(root, width=30, height=6)
editor.pack()
editor.insert("1.0", "Лес\nМоре")

s = editor.get("1.0", "end-1c")
editor.delete("1.0", tk.END)
```

Индекс «строка.столбец»

"1.0" — самый первый символ.
Строки нумеруются с 1, столбцы — с 0.
Индекс передаётся **строкой**, не числом 1.0.

Чтение и очистка

`get("1.0", "end-1c")` читает пользовательский текст. `delete("1.0", tk.END)` очищает его.

Text хранит один дополнительный перевод строки

Фрагмент · editor — созданный Text

```
s = editor.get("1.0", "end-1c")
chars = len(s)
lines = s.count("\n") + 1 if s else 0
```

end-1c убирает только **служебный** последний символ. Переводы строк пользователя остаются.

Текст пользователя	Симв.	Строк
" "	0	0
"Лес"	3	1
"Лес\n"	4	2
"\n"	1	2
"Лес\nMore"	8	2

Перенос длинной строки на экране не добавляет символ \n.

Не применяйте `strip()` или `rstrip()` к тексту, когда пробелы и пустые строки являются данными.

Пауза: предскажите состояние, не запускайте код

1 · Переключатели

Две радиокнопки получили две разные StringVar. Будут ли они взаимоисключающими?

2 · Список

curselection() вернул (0,).
Есть ли выбранная строка?

3 · Многострочный ввод

Пользователь ввёл «Лес» и нажал Enter.
Сколько символов вернёт чтение до end-1с?

4 · Подтверждение

Настройку уже применили, затем поменяли флажок, но не нажали кнопку.
Должен ли измениться прежний итог?

Подумайте. Для каждого ответа назовите не только результат, но и правило, из которого он следует.

Данные записи сохраняем в момент добавления

Обычные данные Python · без GUI

```
books = [  
    {"title": "Маяк", "pages": 120},  
    {"title": "Атлас", "pages": 80},  
]  
total = 0  
for book in books:  
    total += book["pages"]
```

Снимок параметров

Храним значение, снятое с формы, а не ссылку на текущее положение переключателя.

Сводка по сохранённым данным

Общий объём — сумма страниц всех книг. Не текущее поле × число записей.

Две одинаковые записи могут быть разными элементами коллекции. Если дубликаты разрешены, не заменяйте список множеством.

Отдельная функция показывает текущие данные

Фрагмент · books — список словарей, lst — созданный Listbox

```
def redraw(selected=None):  
    lst.delete(0, tk.END)  
    for book in books:  
        line = f"{book['title']} / {book['pages']}"  
        lst.insert(tk.END, line)  
    if selected is not None:  
        lst.selection_set(selected)
```

Изменить данные

Добавить, удалить или переставить запись в books. Затем показать новый список.

Сохранить нужный выбор

При перерисовке выделение исчезает. Передайте новый индекс, когда его нужно восстановить.

В этом фрагменте выбранный индекс предполагается допустимым. None означает «ничего не выделять».

Составное окно — несколько понятных частей

The screenshot shows a window titled "Книжная полка" (Bookshelf). It contains two main frames:

- Frame: форма** (Form): Contains input fields for "Название" (Name) with the value "Маяк" and "Страниц" (Pages) with the value "120". Below these is a blue button labeled "Записать" (Save).
- Frame: коллекция** (Collection): Contains a list of items: "Атлас · 80 стр." and "Маяк · 120 стр.". Below the list is a label "Записей: 2" (Records: 2).

At the bottom of the window, a status bar reads: "Данные отдельно. Элементы окна показывают их." (Data separately. Window elements show them.)

Форма

Собирает текущие значения пользователя.

Коллекция

Хранит записи и порядок.

Отображение

Показывает список, выбор и сводку.

Для небольшого проекта достаточно данных, нескольких коротких обработчиков и функции обновления. Сложная архитектура не обязательна.

Перед кодом: составьте контракт каждого действия

Что изменится? Что должно остаться прежним?

Действие	Изменяем	Сохраняем
Редактировать поле	Текст поля	Уже подтверждённый результат*
Добавить запись	Коллекцию, сводку	Параметры старых записей
Переместить запись	Порядок, индекс выбора	Состав и общую сумму

* Когда по условию результат обновляет только кнопка.

Подумайте. Очистить коллекцию — это то же самое, что сбросить все поля и переключатели?

Типичные ошибки: проверяйте причину

Симптом	Что проверить
Элемент не появился	Есть ли <code>pack/grid</code> и правильный родитель?
Обработчик сработал сразу	Не написано ли <code>command=f ()</code> ?
Каждый клик показывает одно число	Не сбрасывается ли состояние внутри функции?
Сложение склеивает цифры	Выполнено ли преобразование строки в число?
Первая строка списка игнорируется	Не проверяется ли индекс 0 как <code>False</code> ?
Теряются пустые строки	Не удалены ли они через <code>strip/rstrip</code> ?
После переноса выделено другое	Обновлён ли индекс выбранной записи?

Читайте последнюю строку `traceback` и место ошибки. Не исправляйте случайные строки «на всякий случай».

Как сдавать именно наши Tkinter-задачи

На своём компьютере

Один файл .py с полным приложением.
Один Tk и mainloop.
Проверьте настоящее окно: все элементы видны, ничего не перекрывается.

Что пишет ученик

Обычный tkinter.
Не импортирует fake_tk.
Не читает служебный JSON.
Не заменяет GUI вызовами input/print.

В Контесте

Для этой серии выберите
(make) Python 3.12.3.
Отправляется тот же полный .py.

Границы учебной проверки

Используем API из условий. Без ttk, bind, after, диалогов и потоков. Scale — с отдельной кнопкой.

Чекер проверяет поведение, но не всё удобство оформления. Порядок полей и точные тексты берите из условия конкретной задачи.

Теперь можно решать последовательно

Задачи 01–04

Окно → действие → состояние → ввод.

Задачи 05–10

Числа → несколько операций → ошибка
→ память.

Задачи 11–13

Независимый выбор → один вариант →
диапазон.

Задачи 14–16

Список → выбранный элемент → много-
строчный текст.

Задачи 17–20

Снимок формы → группы → порядок →
коллекция записей.

Подумайте. Для своей следующей задачи назовите: данные, действие и наблюдаемый результат.

Собрать → проверить → улучшить

Источники и материалы для повторения

Python и шаблон

P1 · Python 3.12: tkinter
P2 · Python: функции и области видимости
P3 · Python: строки, списки, словари
P4 · Python: обработка исключений
P5 · CPython: Python-обёртки Tkinter
SimplePlus · тема Pin-Yen Huang

Наш контекст

A1 · tkinter_contest_01_10.zip
A2 · tkinter_contest_11_20.zip
Основа покрытия: условия и SUPPORTED_API.md.
Примеры презентации разработаны отдельно.

Справочник Tcl/Tk 8.6

T1 · pack T2 · grid
T3 · Entry T4 · Checkbutton
T5 · Radiobutton T6 · Scale
T7 · Listbox T8 · Text
T9 · Окно T10 · Label
T11 · Button T12 · Frame

В комплекте

12 запускаемых демонстраций.
10 учебных иллюстраций.
Заметки преподавателя и карта тем.